

INSTITUT DE FORMATION AUX NORMES ET TECHNOLOGIES DE L'INFORMATIQUE
SOKODE

Docker

Optimiser le déploiement de vos applications

KONDI Abdoul malik [1]

IFNTI

October 1, 2025

[1] Inspiré de la documentation Docker et du cours d'OpenClassrooms

Table des matières

1. Concepts général
2. Docker
3. Les Conteneurs
4. Les Volumes
5. Les Images
   - Le Dockerfile
   - Manipuler les images
   - Résumé
6. Les Couches (Layers)
   - Expérience 2 : Lecture & écriture
7. Le cache Docker
   - Utilisation du cache
   - Désactivation du cache
   - Importance de l'ordre des instructions
8. Les Réseaux
9. Docker Compose

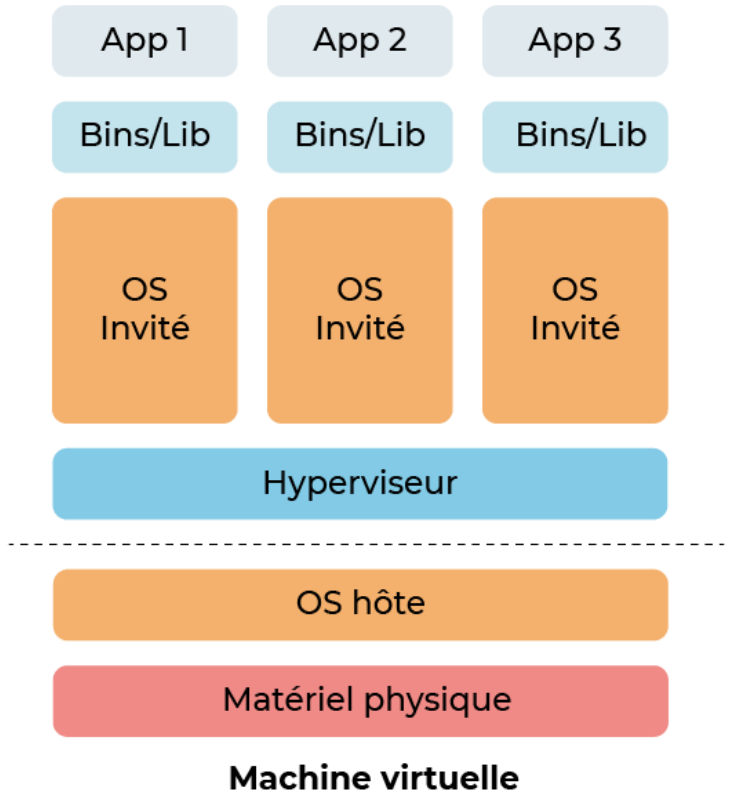
Concepts général

Quelques Notions

Notions

- Machine virtuelle
- Conteneur

Machine virtuelle 1/2



Machine virtuelle

Machine virtuelle 2/2 : Une virtualisation lourde

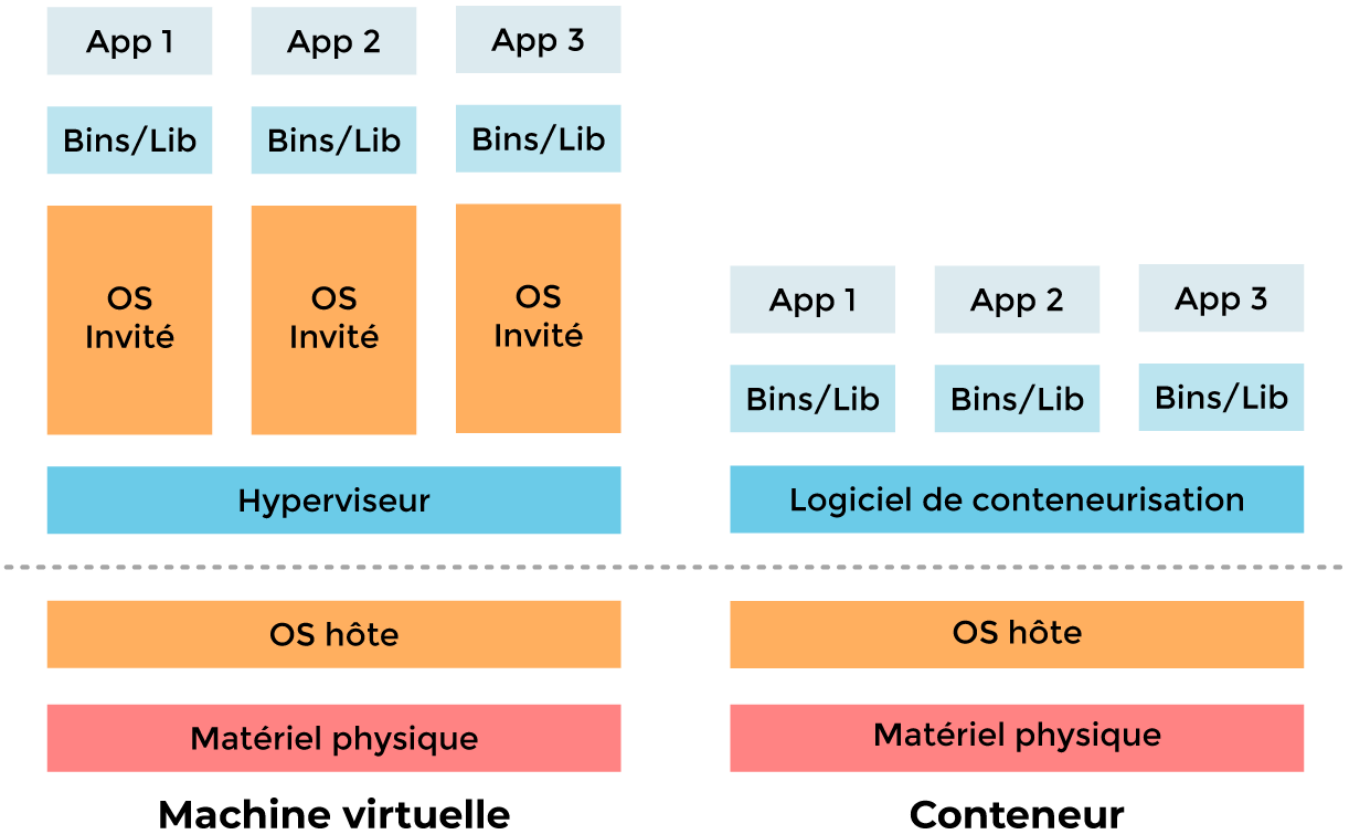
Avantages

- Isolation totale
- Réservation des ressources
- Support de plusieurs OS

Inconvénients

- Démarrage lent
- Réservation des ressources

Conteneur 1/3



Conteneur 2/3 : Une virtualisation légère

Avantages

- Réservation uniquement des ressources nécessaires
- Démarre plus rapidement
- Donne plus d'autonomie aux développeurs

Inconvénients

- Pas d'isolation totale

Conteneur 3/3

Attention

Les conteneurs existaient bien avant Docker. On peut citer OpenVZ et LXC.

Docker vs LXC ou OpenVZ

- Docker : Logiciel de conteneurisation
- LXC ou OpenVZ : Solutions de virtualisation d'OS

Pourquoi utiliser des conteneurs ?

- Réduction des coûts
- Augmentation de la densité de l'infrastructure
- Scalabilité de l'infrastructure

Un nouveau métier 1/2

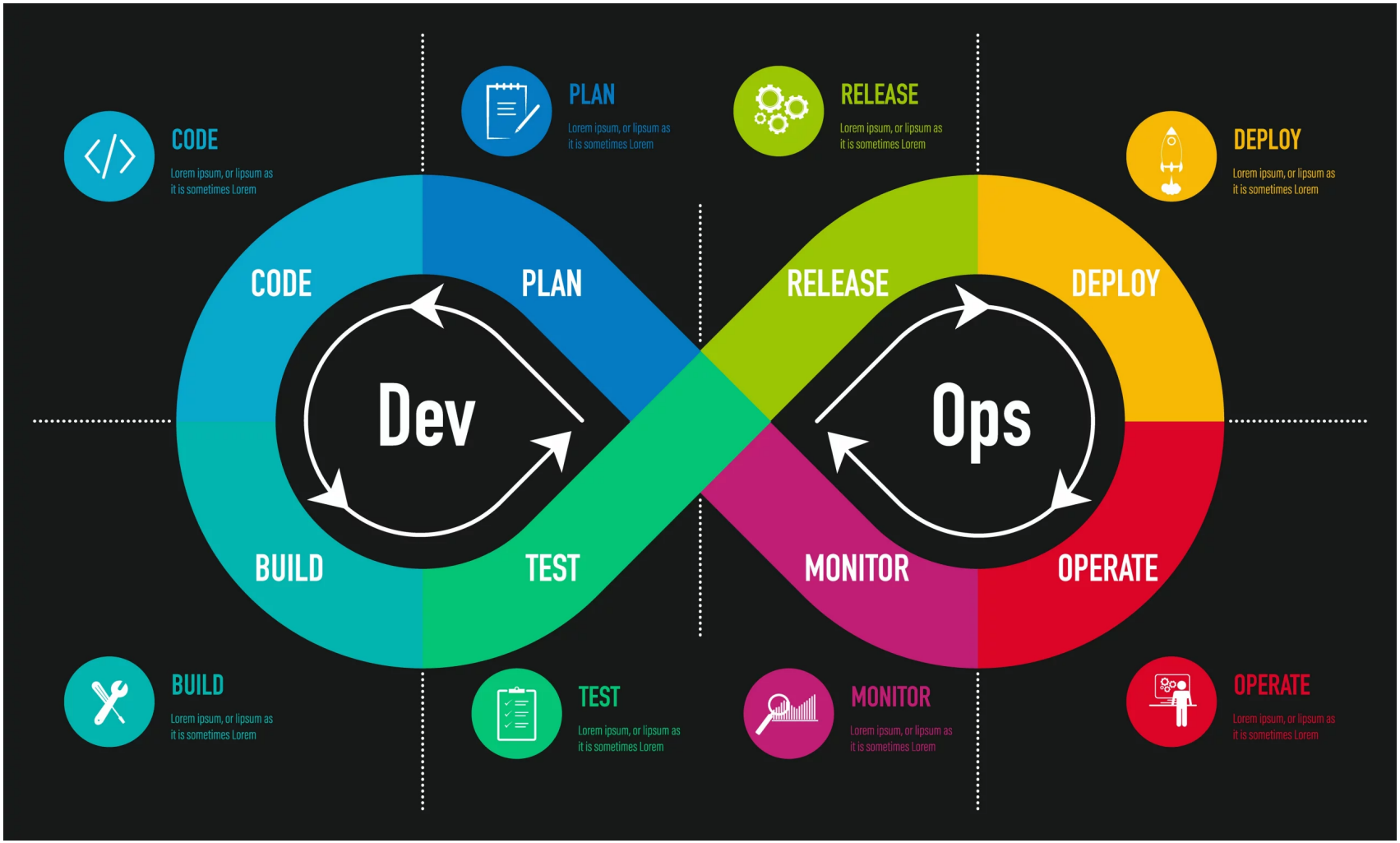
Auparavant

- Les administrateurs système (SysAdmin)
- Les développeurs (Dev)

Exemple d'équation entre ces deux acteurs

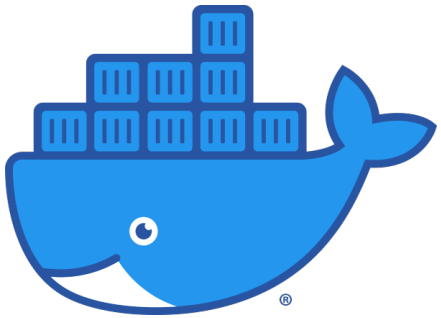
Administrateur système = Garant de la stabilité et de la sécurité des systèmes informatiques
Développeurs = Créateurs de nouvelles fonctionnalités et applications

Un nouveau métier 2/2



Docker

Qu'est ce que docker ?





Docker

- Une plateforme de conteneurisation.
- Créer en 2013 par la société dotCloud.

Objectif

Objectif

Docker permet de faciliter principalement le déploiement des applications web.

Pourquoi docker ?

Raisons

- Éviter les difficultés lier au déploiement de programme.
- Avoir un environnement unifié et fonctionnel.

Comprendre la terminologie de Docker 1/2

Avant de se lancer dans l'utilisation de Docker, il est important de comprendre la terminologie suivante :

Quelques termes

- Image : programme permettant de créer un conteneur.
- Conteneur : instance d'une image.
- Dockerfile : Fichier de configuration permettant de créer une image.
- Docker compose : Fichier de configuration permettant de créer et de gérer plusieurs conteneurs.
- Registry : C'est un dépôt d'image. La registry officielle de Docker s'appelle **docker hub**
- Stack : C'est un ensemble de conteneurs.

Installer docker 1/2

Trois versions proposées par docker Inc

- Docker Community Edition (Linux seulement)
- Docker Desktop (Mac ou Windows)
- Docker Enterprise (Linux seulement et il est propriétaire)

Installer docker 2/2

Commande linux

Lien windows & mac

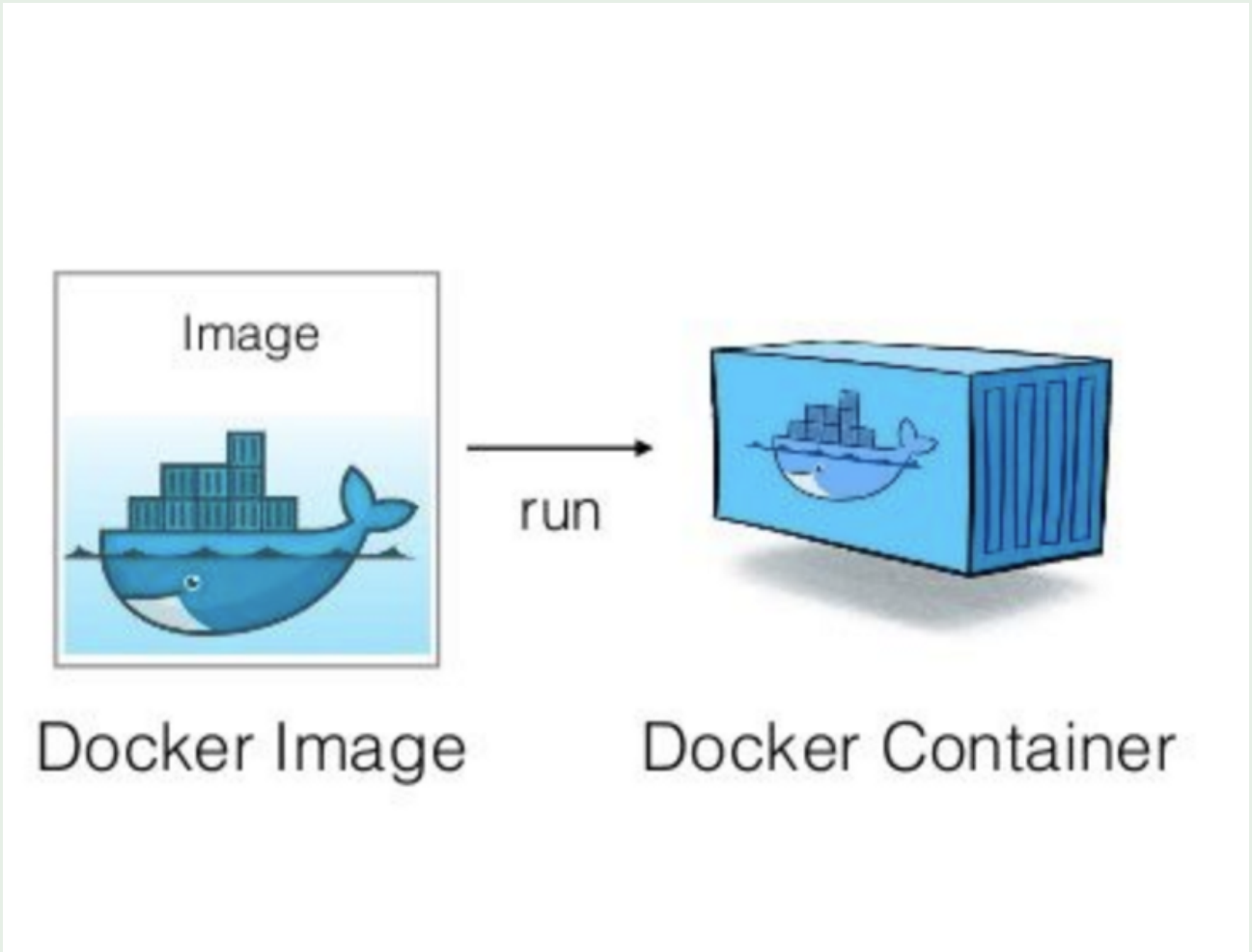
Attention

Pour ceux qui sont sous windows, il est obligatoire de créer un compte docker Hub. Voici le lien *https://hub.docker.com/*

Les Conteneurs

D'abord ...

Qu'est ce que c'est ?



Télécharger une image : hello-world

Que fait chaque commande ?

- docker image ls
- docker pull hello-world[:latest]
- docker image ls

Types de conteneurs

Deux types

- Conteneur persistant ou avec service
- Conteneur éphémère ou jetable

TP 1 : Manipulez vos premiers conteneurs (1/2)

Créer des conteneurs

1- docker container ls
   - Lancez cette commande et dites ce qu'elle fait ?

2- docker run hello-world
   - Lancez cette commande et dites ce qu'elle fait ?

3- docker run nginx
   - Lancez cette commande et dites ce qu'elle fait ?

4- Quelle remarque faites-vous ?

5- docker run -d nginx
   - Lancez cette commande et dites ce qu'elle fait ?

TP 1 : Manipulez vos premiers conteneurs (2/2)

Accéder aux conteneurs

1- De quelle(s) information a-t-on besoin pour accéder à un conteneur ?

2- docker inspect *ID_CONTENEUR*

- Lancez cette commande et dites ce qu'elle fait ?
- Récupérer les informations demandé précédemment.

3- Accéder au conteneur via votre navigateur web.

5- docker run -d -p 8080:80 nginx

- Lancez cette commande et allez à l'adresse suivante : localhost:8080.
- Pouvez vous dire ce que fait cette commande ?

5- docker exec -it *ID_CONTENEUR* bash

- Lancez cette commande et dite ce qu'elle fait ?

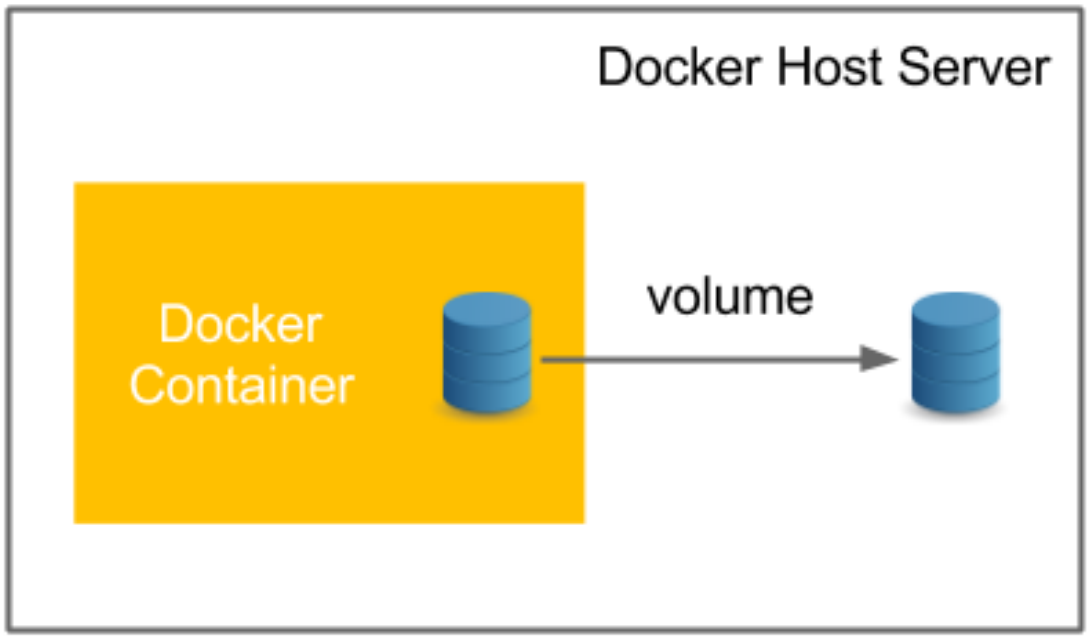
En résumé

Quelques commandes

- docker ps ou docker container ls
- docker run image:version avec les options -d et -p
- docker stop ID_CONTENEUR
- docker start ID_CONTENEUR
- docker exec -it ID_CONTENEUR
- docker rm ID_CONTENEUR avec l' options -f
- docker inspect
- docker system prune

Les Volumes

Introduction



Utilité

- Rendre un conteneur state full

Type de volume

Deux types

- Les volumes persistants
- Docker Volume

TP 2 Les volumes persistants (1/2)

utiliser un volume

1- docker run -d –name serveur_web nginx
   - Lancez cette commande et dites ce qu'elle fait.
   - Afficher la liste de tous les conteneurs et accéder au conteneur serveur_web nginx via le navigateur.

2- Quelle est le chemin du fichier **index.html** du serveur nginx ? Modifier le fichier index.html et réactualiser la page.

3- Supprimer le conteneur

4- Recréer le conteneur serveur_web. vérifiez via le navigateur si le contenu de votre fichier index.html a toujours les modification appliqué. C'est le cas ?

5- Créer un répertoire html. Créer s'y un fichier index.html mettez s'y ce que vous voulez.

TP 2 Les volumes persistants (2/2)

Utiliser un volume

6- docker run -d –name serveur_web -v $(pwd)/html:/usr/share/nginx/html nginx

- Visiter le contenu du fichier index.html via le navigateur.
- Modifier le contenu du fichier index.html et revisité la page html.
- Que fait l'option -**v**.

TP 2 Docker Volume (1/2)

Utiliser docker volume

1- docker volume ls
   - Que fait cette commande ?

2- docker volume create mon_volume.
   - Que fait cette commande ?

3- docker inspect mon_volume.
   - Que fait cette commande ?
   - Dans quelle répertoire le volume à été créé ?

4- docker run -d –name serveur_web –mount source=mon_volume,target=/usr/share/nginx/html nginx
   - Que fait cette commande ?

- Modifier le contenu du fichier index.html et revisiter la page html.

TP 2 Docker Volume (2/2)

Utiliser docker volume

5- Supprimer le conteneur

6- Recréer le conteneur serveur_web (ref *N*-4). vérifier via le navigateur si le contenu de votre fichier index.html à toujours les modifications appliqué. C'est le cas ?

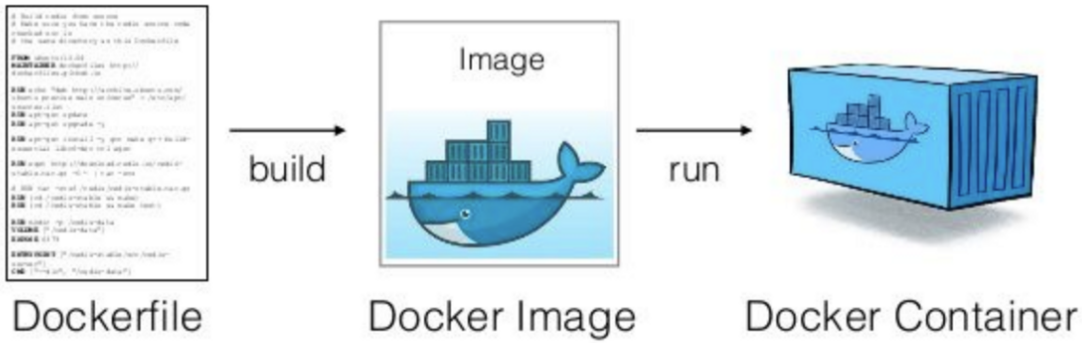
En résumé

Quelques commandes

- docker volumes
- docker run -d –name *nom_conteneur* -v *chemin_volume_hôte*:*chemin_volume_conteneur* *nom_image*
- docker run -d –name *nom_conteneur* -v *nom_volume*:*chemin_volume_conteneur* *nom_image*
- docker run -d –name [nom_conteneur] –mount source=*nom_volume*,target=*chemin_volume_conteneur* *nom_image*

Les Images

Le Dockerfile



C'est quoi ?

Un Dockerfile est un fichier de configuration.

Attention

Chaque ligne du fichier **Dockerfile** créer une couche (layer).

Quelques closes d'un Dockerfile

- FROM *image:version*: Définir l'image de base.
- RUN *commande* : Exécuter une commande système
- ENV *var=valeur* : Définir une variable d'environnement.
- EXPOSE *port* : Exposer un port d'écoute.
- VOLUME *nom_volume* : Définir un volume.
- COPY *src dest* : Copier des données de l'hôte vers le conteneur.
- CMD *["commande", ...]* : Définir le processus maître.

TP 3 : Déployer un site html, css et js



TP 3 : Architecturer des fichiers

```
mon_site_web/
└── gestion_ticket
    ├── index.html
    ├── index.js
    ├── README.md
    └── styles.css

1 directory, 4 files
```

TP 3 : Dockerfile

Instructions

- Créer un répertoire nommé mon_site_web.
- Copier s'y le répertoire gestion_ticket qu'on vas vous fournir.
- Créer un dockerfile dans le répertoire nommé mon_site_web. Voici sont contenu :
  FROM nginx WORKDIR /usr/share/nginx/html/ COPY gestion_ticket/* . ENTRYPOINT ["nginx", "-g", "daemon off;"]

Créer une image

Instructions

- Exécuter la commande : docker image ls.
- docker build -t image_gestion_ticket .
  - Que fait cette commande ? (Exécuté la).
  - Exécuter la commande : docker image ls.
- Créer un conteneur nommé mon_site à partir de l'image image_gestion_ticket.

Les actions possible sur une image.

Quelques commandes

- Pull (lancer docker pull –help)
- Tag (lancer docker tag –help)
- Push (lancer docker push –help)

En résumé

Quelques commandes

- docker build -t [nom_image]:[tag] .
- docker build -t [nom_image][:[tag]] -f nom_dockerfile] .
- docker tag [nom_image]:[tag_actuel] [nom_image]:[nouveau_tag]
- docker pull [nom_image]:[tag]
- docker push [nom_image]:[tag]

Les Couches (Layers)

Types & propriétés

Les types de couches

- Lecture
- Lecture & écriture

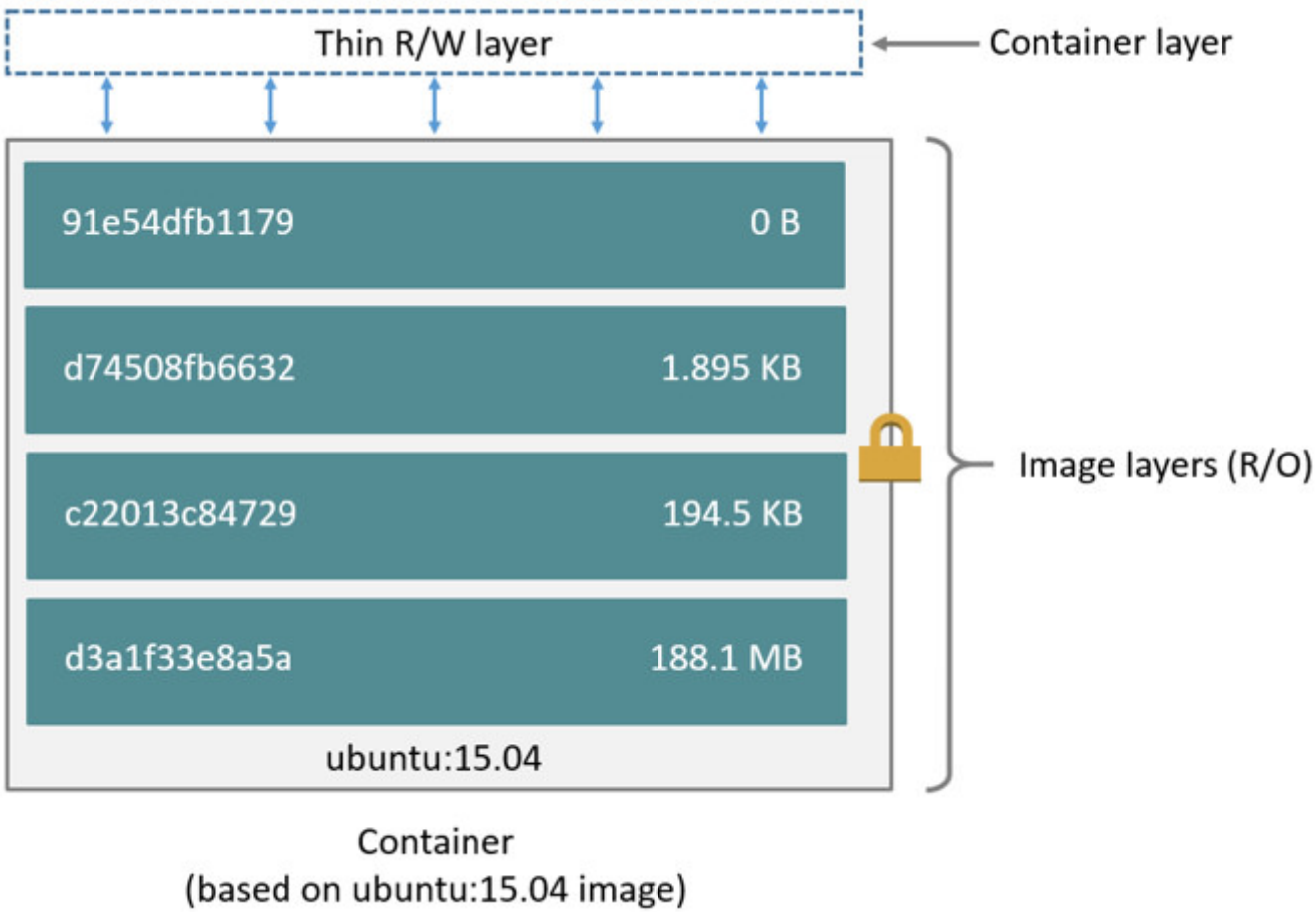
Les propriétés

- partage de couche.

Attention

- Chaque couche est independante.

Principe du Copy & Write



TD Expérience 1 : Lecture (1/3)

Instructions

- Créer un fichier **Dockerfile** avec le contenu suivant :

```
FROM ubuntu:latest
RUN apt-get update
RUN apt-get install -y --no-install-recommends vim
RUN apt-get install -y --no-install-recommends git
RUN apt-get remove git
```

- Construire une nouvelle image à partir de ce fichier.
- Exécuter la commande :

```
docker history ubuntu_image:latest
```

TD Expérience 1 : Lecture (2/3)

Questions

- Que fait cette commande ?
- Que signifie la valeur `<missing>` dans le champ `IMAGE` ?
- Combien de couches ont été créées par cette image ?
- Quel devrait être le poid de cette image ?
- Es ce la cas ?
- pourquoi ?

TD Expérience 1 : Lecture (3/3)

Optimisation du Dockerfile

```
FROM ubuntu:latest
RUN apt-get update \
&& apt-get install -y --no-install-recommends vim git
```

Questions

- Affichez l'historique de l'image construite avec ce nouveau fichier.
- Que constatez-vous ?
- Est-ce normal ? Pourquoi ?

TD Expérience 2 : Lecture & écriture (1/2)

Manipulation du conteneur

- Créer un conteneur à partir de l'image précédente.
- Se connecter au conteneur.
- Créer un fichier `new_file`.
- Supprimer le répertoire `/var`.
- Exécuter cette commande :

  `docker diff <ID_CONTENEUR>`

  Que fait cette commande ?

TD Expérience 2 : Lecture & écriture (2/2)

Création d'une nouvelle image

```
docker commit \
-m "création nouvelle image" \
<ID_CONTENEUR> new_image:version
```

Attention

Cette commande transforme l'état du conteneur modifié en une nouvelle image Docker.

Le cache Docker

But du cache Docker

Objectifs

- Construire plus vite les images
- Démarrer plus vite les conteneurs
- Stocker des images légères
- Partager des couches (layers)

Expérience 1 : Utilisation du cache (1/2)

Dockerfile

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add vim
```

Expérience 1 : Utilisation du cache (1/2)

Dockerfile

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add vim
```

Commandes

```
docker build -t test_vim .
docker build -t test_vim .
```

Observations

Qu'est ce qu'on ob :

- Lors du premier build ?
- Lors du second build ?

Expérience 1 : Utilisation du cache (2/2)

Même avec un nom différent

```
docker build -t test_vim2 .
```

Expérience 1 : Utilisation du cache (2/2)

Même avec un nom différent

```
docker build -t test_vim2 .
```

Conclusion

Une couche n'est téléchargée que si elle n'existe pas déjà.

Expérience 2 : Ne pas utiliser le cache

Dockerfile

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add --no-cache vim
```

Expérience 2 : Ne pas utiliser le cache

Dockerfile

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add --no-cache vim
```

Commande

```
docker build --no-cache -t test_vim2 .
```

Expérience 2 : Ne pas utiliser le cache

Dockerfile

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add --no-cache vim
```

Commande

```
docker build --no-cache -t test_vim2 .
```

Observations

- Le cache est utilisé pour toutes les couches sauf la 3.
- Avec l'option `--no-cache`, aucun cache n'est appliqué.

Expérience 3 : Ordre des instructions

Dockerfile 1

```
FROM alpine:latest
LABEL maintainer="amk"
ENV myvariable toto
RUN apk add vim
```

Dockerfile 2

```
FROM alpine:latest
LABEL maintainer="amk"
ENV myvariable titi
RUN apk add vim
```

Expérience 3 : Ordre des instructions (suite)

Dockerfile 3

```
FROM alpine:latest
LABEL maintainer="amk"
RUN apk add vim
ENV myvariable toto
```

Conclusion

- L'ordre des instructions compte.
- Placer les instructions souvent modifiées en bas du Dockerfile.
- Les instructions stables doivent être en haut pour maximiser l'usage du cache.